Witness 생성 개요

목차

- Witness란 무엇인가
- Witness의 역할
- 전체 생성 파이프라인
- 주요 Witness 타입
- 데이터 흐름
- 구현 구조

Witness란 무엇인가

Witness(증인 데이터)는 영지식 증명 시스템에서 회로(Circuit)에 입력되는 실제 값들을 의미합니다. ZKEVM에서 Witness는 EVM 실행의 모든 상태와 연산을 검증 가능한 형태로 변환한 데이터입니다.

핵심 개념

Witness의 특징:

- **Private Input**: 증명자만 알고 있는 비공개 입력 데이터
- **Structured Data**: 회로의 제약 조건을 만족하도록 구조화된 데이터
- **Complete Information**: 실행을 완전히 재현할 수 있는 모든 정보 포함
- **Verifiable**: 회로의 제약 조건을 통해 검증 가능

Witness vs Public Input:

- Witness: 증명자만 알고 있는 데이터 (예: 트랜잭션 실행 세부사항)
- Public Input: 검증자도 알고 있는 데이터 (예: 블록 해시, 상태 루트)

Witness의 역할

1. 실행 추적 (Execution Trace)

Witness는 EVM 실행의 모든 단계를 추적합니다:

```
// ExecStep: 실행 단계를 나타내는 Witness
pub struct ExecStep {
pub call_index: usize, // 호출 인덱스
pub rw_indices: Vec<(RwTableTag, usize)>, // RW 연산 인덱스
pub execution_state: ExecutionState, // 실행 상태
pub rw_counter: usize, // RW 카운터
pub program_counter: u64, // 프로그램 카운터
pub stack_pointer: usize, // 스택 포인터
pub gas_left: u64, // 남은 가스
pub gas_cost: u64, // 가스 비용
pub memory_size: u64, // 메모리 크기
pub opcode: Option<OpcodeId>, // Opcode
// ...
}
```

2. 상태 변화 기록

모든 읽기/쓰기 연산을 기록합니다:

```
// Rw: Read-Write 연산을 나타내는 Witness
pub enum Rw {
Stack { rw_counter, is_write, call_id, stack_pointer, value },
Memory { rw_counter, is_write, call_id, memory_address, value, value_prev },
AccountStorage { rw_counter, is_write, account_address, storage_key, value, valu
// ... 다른 RW 타입들
}
```

3. 회로 간 데이터 공유

Witness는 여러 회로 간에 데이터를 공유하는 매개체 역할을 합니다:

- **Block Witness**: 블록 메타데이터를 모든 회로에 제공
- **Transaction Witness**: 트랜잭션 데이터를 Tx Circuit과 EVM Circuit에 제공
- **RW Witness**: 상태 변화를 State Circuit과 EVM Circuit에 제공

전체 생성 파이프라인

단계별 프로세스

```
┌──────────────┐
| Geth Trace | ← 1. EVM 실행 트레이스 수집
└──────┬───────┘
|
```

```
▼
┌──────────────────┐
| Bus-Mapping | ← 2. 트레이스를 회로 입력으로 변환
| (CircuitInput | - ExecStep 생성
| Builder) | - RW 연산 생성
└──────────┬───────┘
|
▼
┌──────────────────┐
| Block Convert | ← 3. Block 구조체로 변환
| (block_convert) | - Witness 데이터 구조화
└──────────┬───────┘
|
▼
┌──────────────────┐
| Circuit | ← 4. 각 회로별 Witness 생성
| Witnesses | - EVM Circuit Witness
| | - State Circuit Witness
| | - Bytecode Circuit Witness
| | - 기타 회로 Witnesses
└──────────────────┘
```

1. Geth 트레이스 수집

입력: Ethereum 블록 데이터

출력: `GethExecTrace` - 각 트랜잭션의 실행 트레이스

```
// GethExecTrace 구조 (eth-types에서 정의)
pub struct GethExecTrace {
pub gas: u64,
pub failed: bool,
pub return_value: String,
pub struct_logs: Vec<GethExecStep>,
}
```

2. Bus-Mapping 변환

입력: `GethExecTrace`

출력: `CircuitInputBuilder` - 회로 입력 데이터

`CircuitInputBuilder` 는 트레이스를 분석하여:

- 각 opcode에 대한 `ExecStep` 생성
- 스택, 메모리, 스토리지 연산을 `RwOp` 로 변환
- Copy 이벤트, Keccak 입력 등 수집

```
pub struct CircuitInputBuilder {
pub sdb: StateDB, // 상태 데이터베이스
pub code_db: CodeDB, // 코드 데이터베이스
pub block: Blocks, // 블록 데이터
pub block_ctx: BlockContext, // 블록 컨텍스트
```

```
}
```

3. Block 변환

입력: `CircuitInputBuilder`

출력: `Block` - 모든 회로가 사용하는 통합 Witness 구조

`block_convert` 함수가 수행하는 작업:

```
pub fn block_convert(
block: &circuit_input_builder::Blocks,
code_db: &CodeDB,
) -> Result<Block, Error> {
// 1. RW 연산 맵 생성
let rws = RwMap::from(&block.container);
// 2. 트랜잭션 변환
let txs = block.txs().iter().map(|tx| tx_convert(tx, ...)).collect();
// 3. 실행 단계 변환
let padding_step = step_convert(&block.block_steps.padding_step, ...);
let end_block_step = step_convert(&block.block_steps.end_block_step, ...);
// 4. MPT 업데이트 생성
let mpt_updates = MptUpdates::from_unsorted_rws_with_state_roots(...);
// 5. Bytecode 수집
let bytecodes = get_bytecodes(code_db);
// 6. Block 구조체 생성
```

```
Ok(Block {
context: BlockContexts::from(block),
rws,
txs,
bytecodes,
copy_events: block.copy_events.clone(),
exp_events: block.exp_events.clone(),
sha3_inputs: block.sha3_inputs.clone(),
mpt_updates,
precompile_events: block.precompile_events.clone(),
// ...
})
}
```

4. 회로별 Witness 할당

각 회로는 `Block` 에서 필요한 데이터를 추출하여 자신의 테이블에 할당합니다.

주요 Witness 타입

Block Witness

블록 전체의 컨텍스트와 메타데이터를 담고 있습니다.

```
pub struct Block {
```

```rust
// 트랜잭션 데이터
pub txs: Vec<Transaction>,
pub sigs: Vec<Signature>,
// 실행 단계
pub padding_step: ExecStep,
pub end_block_step: ExecStep,
// RW 연산
pub rws: RwMap,
// 바이트코드
pub bytecodes: BTreeMap<Word, Bytecode>,
// 블록 컨텍스트
pub context: BlockContexts,
// 이벤트 데이터
pub copy_events: Vec<CopyEvent>,
pub exp_events: Vec<ExpEvent>,
pub sha3_inputs: Vec<Vec<u8>>,
// MPT 업데이트
pub mpt_updates: MptUpdates,
// 프리컴파일 이벤트
pub precompile_events: PrecompileEvents,
// 회로 파라미터
pub circuits_params: CircuitsParams,
```

```
}
```

역할:

- 모든 회로가 공유하는 중앙 데이터 저장소
- 회로 간 일관성 보장
- 증명 생성에 필요한 모든 정보 포함

Transaction Witness

개별 트랜잭션의 데이터를 담고 있습니다.

```
pub struct Transaction {
pub block_number: u64,
pub id: usize,
pub hash: H256,
pub tx_type: TxType,
pub nonce: u64,
pub gas: u64,
pub gas_price: Word,
pub caller_address: Address,
pub callee_address: Option<Address>,
pub is_create: bool,
pub value: Word,
pub call_data: Vec<u8>,
pub rlp_unsigned: Vec<u8>,
```

```
pub rlp_signed: Vec<u8>,
pub v: u64,
pub r: Word,
pub s: Word,
pub calls: Vec<Call>,
pub steps: Vec<ExecStep>,
// ...
}
```

역할:

- Tx Circuit에서 트랜잭션 검증
- EVM Circuit에서 실행 추적
- Sig Circuit에서 서명 검증

ExecStep Witness

EVM 실행의 각 단계를 나타냅니다.

```
pub struct ExecStep {
pub call_index: usize,
pub rw_indices: Vec<(RwTableTag, usize)>,
pub execution_state: ExecutionState,
pub rw_counter: usize,
pub program_counter: u64,
pub stack_pointer: usize,
```

```
pub gas_left: u64,
pub gas_cost: u64,
pub memory_size: u64,
pub reversible_write_counter: usize,
pub log_id: usize,
pub opcode: Option<OpcodeId>,
pub block_num: u64,
}
```

역할:

- EVM Circuit에서 각 opcode 실행 검증
- 상태 전이 추적
- 가스 소비 계산

RW Witness

모든 읽기/쓰기 연산을 기록합니다.

```
pub enum Rw {
Start { rw_counter },
Stack { rw_counter, is_write, call_id, stack_pointer, value },
Memory { rw_counter, is_write, call_id, memory_address, value, value_prev },
AccountStorage { rw_counter, is_write, account_address, storage_key, value, valu
TxAccessListAccount { rw_counter, is_write, tx_id, account_address, is_warm, is_
Account { rw_counter, is_write, account_address, field_tag, value, value_prev },
```

```
CallContext { rw_counter, is_write, call_id, field_tag, value },
TxLog { rw_counter, is_write, tx_id, log_id, field_tag, index, value },
TxReceipt { rw_counter, is_write, tx_id, field_tag, value },
// ...
}
```

역할:

- State Circuit에서 상태 일관성 검증
- EVM Circuit에서 연산 검증
- 시간 순서대로 정렬되어 상태 전이 추적

Bytecode Witness

컨트랙트 바이트코드 정보를 담고 있습니다.

```
pub struct Bytecode {
pub hash: Word,
pub bytes: Vec<u8>,
}
```

역할:

- Bytecode Circuit에서 바이트코드 검증
- EVM Circuit에서 opcode 실행 검증
- Code hash 계산 및 검증

데이터 흐름

Block → Circuit Witnesses 흐름

```
Block
├─→ EVM Circuit
| ├─ txs[].steps[] (ExecStep)
| ├─ rws (Rw operations)
| └─ bytecodes (Bytecode)
|
├─→ State Circuit
| └─ rws (Rw operations sorted)
|
├─→ Bytecode Circuit
| └─ bytecodes (Bytecode)
|
├─→ Tx Circuit
| ├─ txs[] (Transaction)
| └─ sigs[] (Signature)
|
├─→ Sig Circuit
| └─ sigs[] (Signature)
|
├─→ Copy Circuit
| └─ copy_events[] (CopyEvent)
```

```
│
├──→ Keccak Circuit
│  └─ sha3_inputs[] (Keccak inputs)
│
├──→ Exp Circuit
│  └─ exp_events[] (ExpEvent)
│
└──→ MPT Circuit
└─ mpt_updates (MptUpdates)
```

Witness 생성 순서

1. **Geth 트레이스 수집**: `GethExecTrace` 획득
2. **Bus-Mapping**: `CircuitInputBuilder` 로 변환
   - Opcode별 `ExecStep` 생성
   - RW 연산 생성
   - 이벤트 수집
3. **Block 변환**: `block_convert` 로 `Block` 생성
   - RW 맵 구성
   - 트랜잭션 변환
   - MPT 업데이트 생성
4. **회로별 할당**: 각 회로가 필요한 데이터 추출
   - `table_assignments()` 메서드 호출
   - Challenge 값 적용

- 셀에 값 할당

구현 구조

주요 모듈

`zkevm-circuits/src/witness/` :

- `block.rs` : Block 구조체 및 변환 함수
- `tx.rs` : Transaction 구조체 및 변환 함수
- `step.rs` : ExecStep 구조체 및 변환 함수
- `rw.rs` : Rw 열거형 및 RwMap
- `bytecode.rs` : Bytecode 구조체
- `call.rs` : Call 구조체
- `mpt.rs` : MPT 업데이트 관련
- `keccak.rs` : Keccak 입력 처리

핵심 함수

block_convert

`CircuitInputBuilder` 를 `Block` 으로 변환하는 메인 함수:

```
pub fn block_convert(
block: &circuit_input_builder::Blocks,
code_db: &CodeDB,
) -> Result<Block, Error>
```

처리 과정:

1. RW 연산 검증 및 맵 생성
2. 트랜잭션 변환 (`tx_convert`)
3. 실행 단계 변환 (`step_convert`)
4. MPT 업데이트 생성
5. Bytecode 수집
6. Block 구조체 조립

tx_convert

트랜잭션을 Witness 형식으로 변환:

```
pub fn tx_convert(
tx: &circuit_input_builder::Transaction,
tx_id: usize,
chain_id: u64,
next_block_num: u64,
) -> Transaction
```

처리 과정:

1. 트랜잭션 필드 추출
2. RLP 인코딩 생성
3. 서명 데이터 변환
4. Call 및 ExecStep 변환

step_convert

실행 단계를 Witness 형식으로 변환:

```
pub fn step_convert(
step: &circuit_input_builder::ExecStep,
block_num: u64,
) -> ExecStep
```

처리 과정:

1. RW 인덱스 변환
2. Execution state 결정
3. 가스 및 메모리 정보 추출
4. Opcode 정보 설정

RwMap 구조

RW 연산을 효율적으로 관리하는 자료구조:

```
pub struct RwMap(pub HashMap<RwTableTag, Vec<Rw>>);
```

주요 메서드:

- `table_assignments()` : 정렬된 RW 연산 리스트 반환
- `table_assignments_unsorted()` : 정렬되지 않은 RW 연산 리스트
- `check_value()` : RW 연산 값 검증
- `check_rw_counter_sanity()` : RW 카운터 연속성 검증

Witness 할당 패턴

각 회로는 다음 패턴으로 Witness를 할당합니다:

```
impl<F: Field> Circuit<F> for MyCircuit<F> {
fn synthesize(
&self,
config: Self::Config,
mut layouter: impl Layouter<F>,
) -> Result<(), Error> {
// 1. Challenge 값 획득
let challenges = Challenges::mock(Value::known(F::from(0x100)));
// 2. Block에서 필요한 데이터 추출
let witness_data = self.block.get_witness_data();
// 3. 테이블 할당
layouter.assign_region(
|| "assign witness",
|mut region| {
for (offset, row) in witness_data.iter().enumerate() {
// 각 셀에 값 할당
region.assign_advice(
|| "value",
config.value,
offset,
|| row.value_assignment(challenges),
```

```
)?;
}
Ok(())
},
)?;
Ok(())
}
}
```

Witness 검증

값 일관성 검증

`RwMap::check_value()` 는 RW 연산의 값 일관성을 검증합니다:

```
pub fn check_value(&self) -> Result<(), Error> {
// 각 RW 그룹에 대해
for group in self.table_assignments_with_idx().values() {
let first = group.first();
// 첫 번째 읽기는 초기값이어야 함
if !first.is_write() && first.value_word() != U256::zero() {
return Err(Error::InternalError("first access reads don't change value"));
}
// 이후 읽기는 이전 값과 같아야 함
```

```
for rw in group.iter().skip(1) {
if !rw.is_write() && rw.value_word() != prev.value_word() {
return Err(Error::InternalError("non-first access reads don't change value"));
}
}
}
Ok(())
}
```

RW 카운터 검증

`RwMap::check_rw_counter_sanity()` 는 RW 카운터의 연속성을 검증합니다:

```
pub fn check_rw_counter_sanity(&self) {
for (idx, rw_counter) in self.all_rws().map(|r| r.rw_counter()).sorted().enumer
debug_assert_eq!(idx, rw_counter - 1);
}
}
```

최적화 기법

1. 병렬 처리

RW 값 검증은 병렬로 수행됩니다:

```
let errs = rows
.into_values()
.par_bridge() // Rayon 병렬 처리
.flat_map(|group| {
// 각 그룹을 병렬로 검증
validate_group(group)
})
.collect();
```

2. 캐싱

정렬 키를 캐싱하여 성능을 향상시킵니다:

```
rows.sort_by_cached_key(Rw::as_key);
```

3. 사전 할당

예상 크기로 HashMap을 사전 할당합니다:

```
let mut map = HashMap::with_capacity(total_len / 22);
```

다음 단계

Witness 생성의 세부 사항을 이해하기 위해 다음 문서들을 참고하세요:

- Block Witness: Block 구조체와 블록 데이터 수집

- Tx Witness: Transaction 구조체와 트랜잭션 데이터 변환
- Step Witness: ExecStep 구조체와 실행 단계 데이터 생성
- 할당 프로세스: Witness를 회로에 할당하는 과정

참고 자료

관련 코드:

- `zkevm-circuits/src/witness/block.rs` : Block 구조체 및 변환
- `zkevm-circuits/src/witness/tx.rs` : Transaction 변환
- `zkevm-circuits/src/witness/step.rs` : ExecStep 변환
- `zkevm-circuits/src/witness/rw.rs` : RW 연산 관리
- `bus-mapping/src/circuit_input_builder.rs` : CircuitInputBuilder

관련 문서:

- Bus-Mapping 개요
- RW 연산
- 데이터 흐름